CSA3710 SOFTWARE TESTING
EXPERIMENTS

QUESTIONS:

16. To check whether a string is an anagram of another string and validate output using JUnit assertions.

Procedure:

1. Create a Java project named AnagramJUnitTest in Eclipse IDE.
2. Add the JUnit 5 library to the project's build path.
3. Create a Java class named AnagramChecker.java and implement the method to check whether two strings are anagrams.
4. Validate null inputs, convert strings to lowercase, remove whitespace, compare lengths, sort character arrays, and compare the sorted arrays.
5. Create a test class named AnagramCheckerTest.java.
6. Write JUnit test cases using @Test and assertion methods such as assertTrue(), assertFalse(), and assertThrows().
7. Test valid anagrams, non-anagrams, different-length strings, case insensitivity, whitespace handling, repeated characters, empty strings, and null inputs.
8. Execute the test class using Run As → JUnit Test.
9. Verify the results in the JUnit view and check whether all test cases pass.
10. Analyze statement coverage, branch coverage, and boundary conditions to evaluate the correctness of the program.

Report Table:

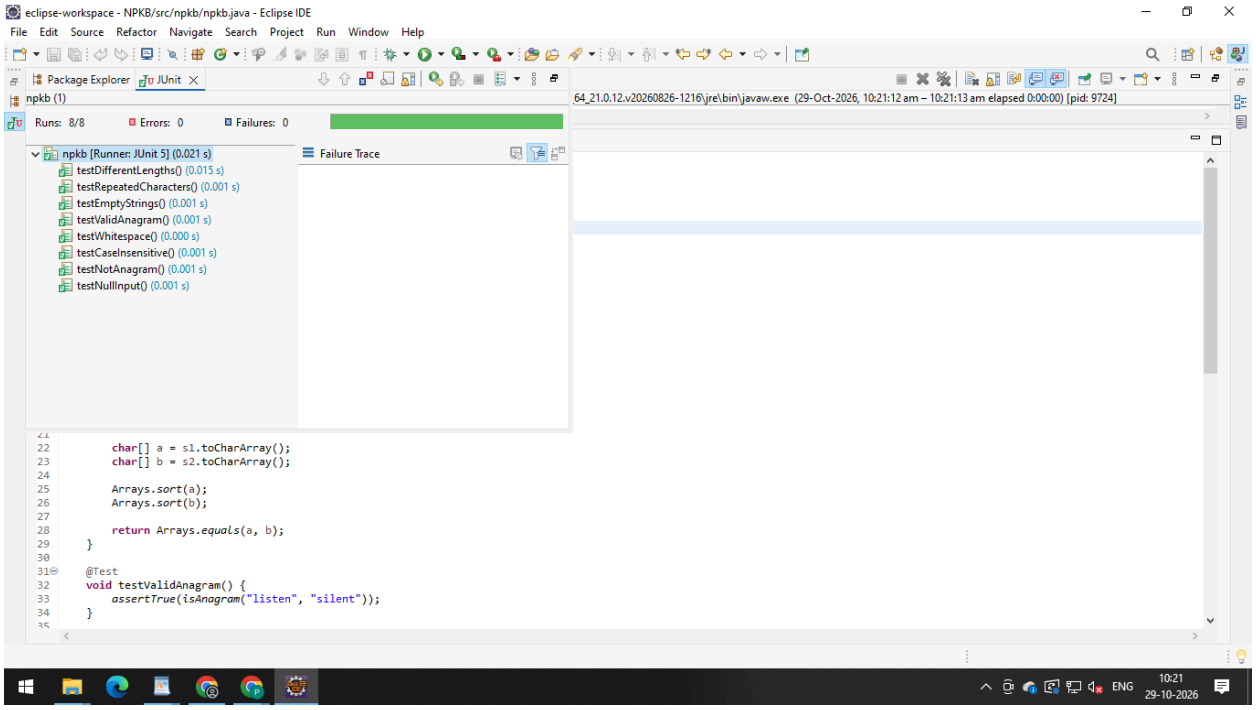
S.No.	Test Case	Input	Expected Result
1	Valid anagrams	listen, silent	True
2	Non-anagrams	hello, world	False
3	Different lengths	cat, cats	False
4	Case-insensitive check	Listen, Silent	True
5	Whitespace handling	dormitory, dirty room	True
6	Repeated characters	aabb, bbaa	True
7	Different character frequencies	aabb, abbb	False
8	Empty strings	"", ""	True
9	Null input	null, abc	Exception
10	Null input	abc, null	Exception

Code:

```
package npkb;
import java.util.Arrays;
import java.util.Locale;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class npkb {
  public static boolean isAnagram(String s1, String s2) {
    if (s1 == null || s2 == null) {
      throw new IllegalArgumentException("Input cannot be null");
    }
    s1 = s1.replaceAll("\\s+", "").toLowerCase(Locale.ROOT);
    s2 = s2.replaceAll("\\s+", "").toLowerCase(Locale.ROOT);
    if (s1.length() != s2.length()) {
```

```
            return false;
        }
        char[] a = s1.toCharArray();
        char[] b = s2.toCharArray();
        Arrays.sort(a);
        Arrays.sort(b);
        return Arrays.equals(a, b);
    }
    @Test
    void testValidAnagram() {
        assertTrue(isAnagram("listen", "silent"));
    }
    @Test
    void testNotAnagram() {
        assertFalse(isAnagram("hello", "world"));
    }
    @Test
    void testDifferentLengths() {
        assertFalse(isAnagram("cat", "cats"));
    }
    @Test
    void testCaseInsensitive() {
        assertTrue(isAnagram("Listen", "Silent"));
    }
    @Test
    void testWhitespace() {
        assertTrue(isAnagram("dormitory", "dirty room"));
    }
    @Test
    void testRepeatedCharacters() {
        assertTrue(isAnagram("aabb", "bbaa"));
        assertFalse(isAnagram("aabb", "abbb"));
    }
    @Test
    void testEmptyStrings() {
        assertTrue(isAnagram("", ""));
    }
    @Test
    void testNullInput() {
        assertThrows(IllegalArgumentException.class,
                () -> isAnagram(null, "abc"));
        assertThrows(IllegalArgumentException.class,
                () -> isAnagram("abc", null));
    }
}
```

Output:


eclipse-workspace - NPKB/src/npkb/npkb.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer
JUnit
npkb (1)
Runs: 8/8
Errors: 0
Failures: 0
npkb [Runner: JUnit 5] (0.021 s)
testDifferentLengths() (0.015 s)
testRepeatedCharacters() (0.001 s)
testEmptyStrings() (0.001 s)
testValidAnagram() (0.001 s)
testWhitespace() (0.000 s)
testCaseInsensitive() (0.001 s)
testNotAnagram() (0.001 s)
testNullInput() (0.001 s)
Failure Trace
64_21.0.12.v20260826-1216\jre\bin\javaw.exe (29-Oct-2026, 10:21:12 am – 10:21:13 am elapsed 0:00:00) [pid: 9724]
char[] a = s1.toCharArray();
char[] b = s2.toCharArray();
Arrays.sort(a);
Arrays.sort(b);
return Arrays.equals(a, b);
}
@Test
void testValidAnagram() {
assertTrue(isAnagram("listen", "silent"));
}
ENG
10:21
29-10-2026

17. To find the maximum of three numbers and validate output using JUnit assertions.

Code:

Maximum:

```
package maximum;
public class Maximum {
		public static int findMax(int a, int b, int c) {
		   return Math.max(a, Math.max(b, c));
		}

}
```

Maximum Test:

```
package maximum;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class MaximumTest {
  @Test
  void testMaximum() {
     assertEquals(9, Maximum.findMax(3, 9, 5));
     assertEquals(10, Maximum.findMax(10, 2, 4));
     assertEquals(7, Maximum.findMax(1, 2, 7));
     assertEquals(5, Maximum.findMax(5, 5, 2));
  }
}
```

Report:

S. No.	Test Case	Input(a,b,c)	Expected Result
1	Maximum is the second number	(3,9,5)	9
2	Maximum is the first number	(10, 2, 4)	10
3	Maximum is the third number	(1, 2, 7)	7
4	Two numbers are equal	(5, 5, 2)	5

Code output:



Maximum.java | MaximumTest.java

```java
package maximum;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class MaximumTest {
    @Test
    void testMaximum() {
        assertEquals(9, Maximum.findMax(3, 9, 5));
        assertEquals(10, Maximum.findMax(10, 2, 4));
        assertEquals(7, Maximum.findMax(1, 2, 7));
        assertEquals(5, Maximum.findMax(5, 5, 2));
    }
}
```

Problems | Servers | Terminal | Data Source Explorer | Properties | Console

<terminated> Bankingnegative [Java Application] C:\Users\SAIL\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win

```
Banking Application
Negative Test Cases
------------------------------
Invalid Login Test: PASS
Insufficient Balance Test: PASS
Invalid Deposit Test: PASS
Invalid Transfer Test: PASS
Empty Username Test: PASS
------------------------------
Testing Completed
```

18. To check whether a number is even or odd and validate output using JUnit assertions.

Code:

```
package npkb;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class npkb {
  public String checkEvenOdd(int n) {
    if (n % 2 == 0)
      return "Even";
    else
      return "Odd";
  }
  @Test
  public void testEvenNumber() {
    assertEquals("Even", checkEvenOdd(10));
  }
  @Test
  public void testOddNumber() {
    assertEquals("Odd", checkEvenOdd(15));
  }
  @Test
  public void testZero() {
    assertEquals("Even", checkEvenOdd(0));
  }
}
```

Report:

S. No.	Test Scenario	Input(N)	Expected Result	JUnit Assertion
1.	Even number	10	Even	assertEquals("Even", checkEvenOdd(10))
2	Odd number	15	odd	assertEquals("Odd", checkEvenOdd(15))
3	zero	0	even	assertEquals("Even", checkEvenOdd(0))

Note:

JUnit assertions are used to verify the expected output.

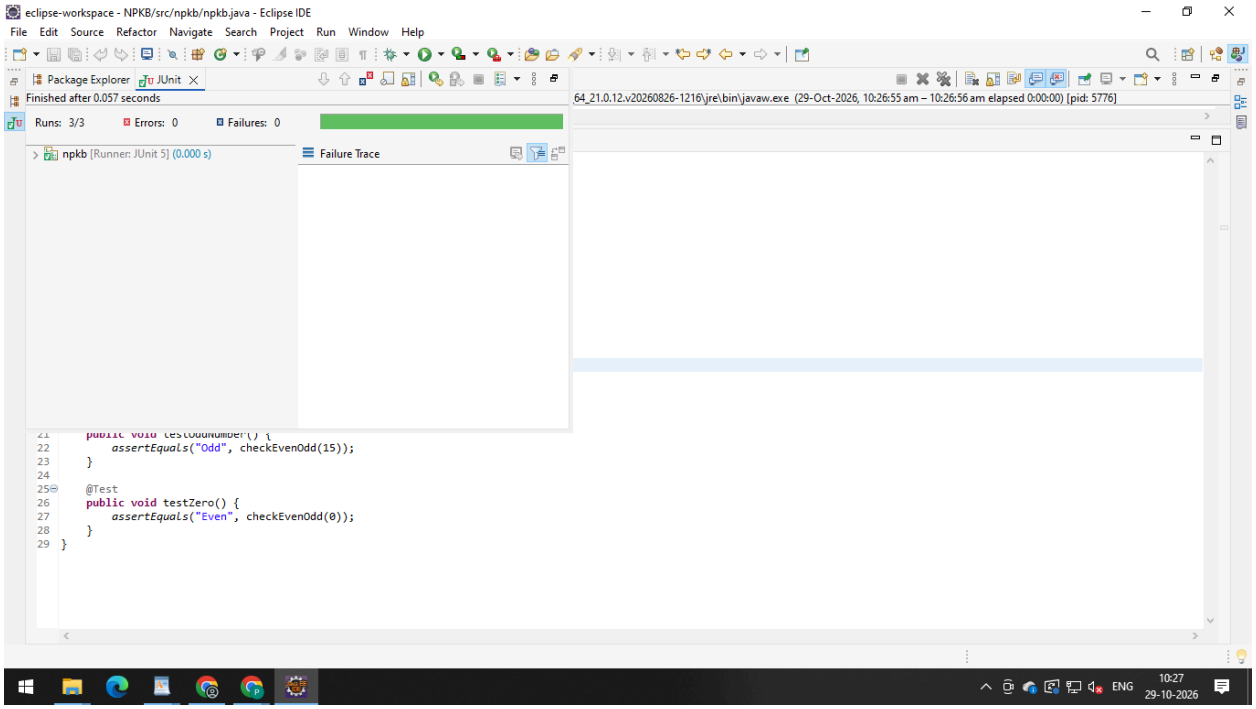
Even numbers are divisible by 2, while odd numbers are not.

Zero is considered an even number.

White-box testing verifies both branches of the `if-else` statement.

Result: Thus, the even-or-odd checking program was executed successfully, and all three test cases were validated using JUnit assertions.

Code output:


eclipse-workspace - NPKB/src/npkb/npkb.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Package Explorer
JUnit
Finished after 0.057 seconds
Runs: 3/3
Errors: 0
Failures: 0
npkb [Runner: JUnit 5] (0.000 s)
Failure Trace
64_21.0.12.v20260826-1216\jre\bin\javaw.exe (29-Oct-2026, 10:26:55 am – 10:26:56 am elapsed 0:00:00) [pid: 5776]
assertEquals("Odd", checkEvenOdd(15));
}
@Test
public void testZero() {
assertEquals("Even", checkEvenOdd(0));
}
}
ENG
10:27
29-10-2026

19. To calculate compound interest and validate output using JUnit assertions.

Code:

CompoundInterest:

```
package maximum;
public class CompoundInterest {
  public static double calculate(double p, double r, int n) {
    return p * Math.pow(1 + r / 100, n) - p;
  }
}
```

CompoundInterestTest:

```
package maximum;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class CompoundInterestTest {
  @Test
  void testInterest() {
    assertEquals(210.0,
      CompoundInterest.calculate(1000, 10, 2), 0.01);
    assertEquals(100.0,
      CompoundInterest.calculate(1000, 10, 1), 0.01);
    assertEquals(0.0,
      CompoundInterest.calculate(1000, 0, 2), 0.01);
  }
}
```

Report Table:

S.No.	Principal (₹)	Rate (%)	Time (Years)	Expected Interest (₹)
1	1000	10	2	210
2	1000	10	1	100
3	1000	0	2	0

Code Output:

Maximum.java | MaximumTest.java | CompoundInterest.java

```
1  package maximum;
2
3  public class CompoundInterest {
4      public static double calculate(double p, double r, int n) {
5          return p * Math.pow(1 + r / 100, n) - p;
6      }
7  }
```

Problems | Servers | Terminal | Data Source Explorer | Properties | Console

```
<terminated> Bankingnegative [Java Application] C:\Users\SAIL\.p2\pool\plugins\org.eclipse.justj.openjdk
Banking Application
Negative Test Cases
------------------------------
Invalid Login Test: PASS
Insufficient Balance Test: PASS
Invalid Deposit Test: PASS
Invalid Transfer Test: PASS
Empty Username Test: PASS
------------------------------
Testing Completed
```

Maximum.java | MaximumTest.java | CompoundInterest.java | CompoundInterestTest.java

```
1  package maximum;
2
3  import org.junit.jupiter.api.Test;
4  import static org.junit.jupiter.api.Assertions.*;
5
6  public class CompoundInterestTest {
7
8      @Test
9      void testInterest() {
10         assertEquals(210.0,
11             CompoundInterest.calculate(1000, 10, 2), 0.01);
12
13         assertEquals(100.0,
14             CompoundInterest.calculate(1000, 10, 1), 0.01);
15
16         assertEquals(0.0,
17             CompoundInterest.calculate(1000, 0, 2), 0.01);
18     }
19 }
```

Problems | Servers | Terminal | Data Source Explorer | Properties | Console

```
<terminated> Bankingnegative [Java Application] C:\Users\SAIL\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86
Banking Application
Negative Test Cases
------------------------------
Invalid Login Test: PASS
Insufficient Balance Test: PASS
Invalid Deposit Test: PASS
Invalid Transfer Test: PASS
Empty Username Test: PASS
------------------------------
Testing Completed
```

20. To count the number of vowels and consonants in a given string and validate output using JUnit assertions.

Code:

```
package junit;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class VowelConsonantCountTest {

   public int[] countVowelsConsonants(String s) {
      int vowels = 0, consonants = 0;
      s = s.toLowerCase();

      for (int i = 0; i < s.length(); i++) {
         char ch = s.charAt(i);

         if (ch >= 'a' && ch <= 'z') {
            if (ch == 'a' || ch == 'e' || ch == 'i'
                  || ch == 'o' || ch == 'u')
               vowels++;
            else
               consonants++;
         }
      }
      return new int[]{vowels, consonants};
   }

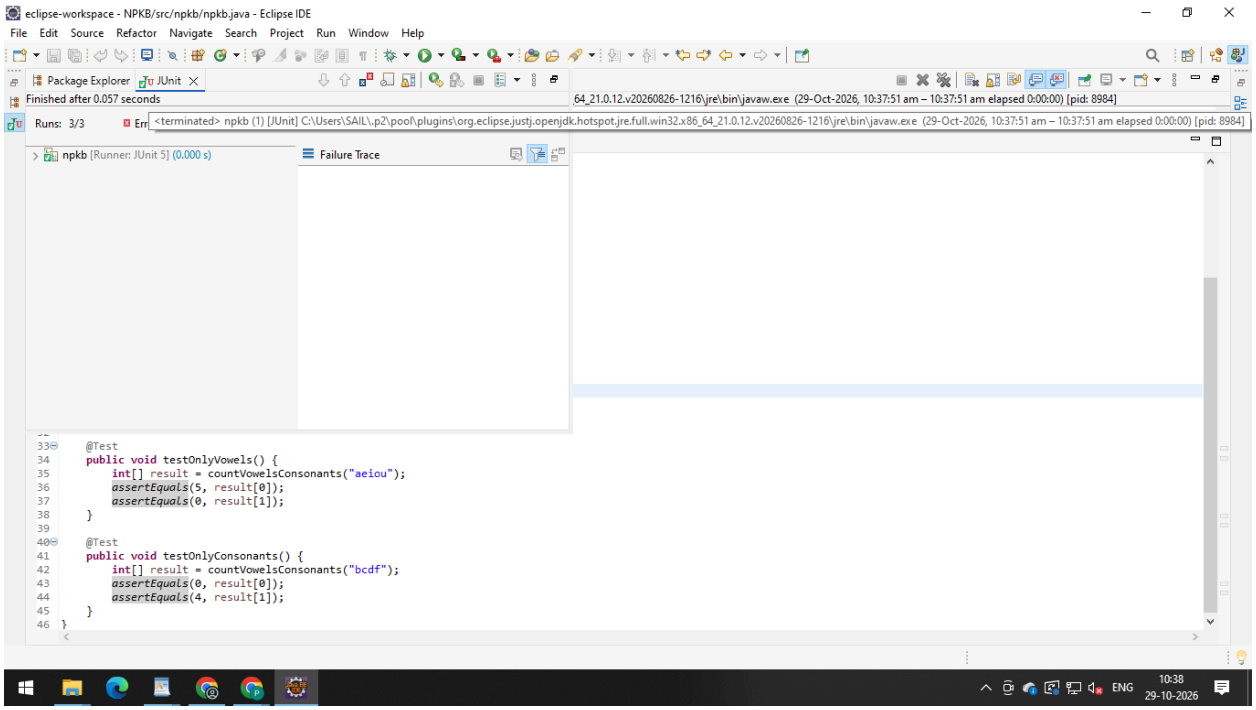
   @Test
   public void testVowelConsonantCount() {
      int[] result = countVowelsConsonants("hello");
      assertEquals(2, result[0]);
```

```
        assertEquals(3, result[1]);
    }

    @Test
    public void testOnlyVowels() {
        int[] result = countVowelsConsonants("aeiou");
        assertEquals(5, result[0]);
        assertEquals(0, result[1]);
    }

    @Test
    public void testOnlyConsonants() {
        int[] result = countVowelsConsonants("bcdf");
        assertEquals(0, result[0]);
        assertEquals(4, result[1]);
    }
}
```

Output:



Report Table:

S. No.	Test Scenario	Input	Expected Result	JUnit Assertion
1	Count vowels and consonants	`hello`	Vowels = 2, Consonants = 3	`assertEquals(2, result[0])`, `assertEquals(3, result[1])`
2	Only vowels	aeiou	Vowels = 5, Consonants = 0	`assertEquals(5, result[0])`, `assertEquals(0, result[1])`
3	Only consonants	bcdf	Vowels = 0, Consonants = 4	`assertEquals(0, result[0])`, `assertEquals(4, result[1])`